



۱۳۰۷

دانشگاه صنعتی خواجه نصیرالدین طوسی

دانشگاه صنعتی خواجه نصیرالدین طوسی

دانشکده مهندسی کامپیوتر

گزارش پروژه درس پایگاه داده‌ها

طراحی و پیاده‌سازی سامانه رزرو و خرید بلیت مسابقات ورزشی

استاد درس: خانم دکتر پیشگو

دستیار ارشد: مهندس علیرضا قربانی

الحمد لله الذي
خلقنا من
الحمم

این گزارش فرآیند طراحی و پیاده‌سازی بک‌اند و زیرساخت داده‌ای سامانه رزرو و خرید بلیت مسابقات ورزشی را تشریح می‌کند. بک‌اند با Django REST Framework و Django پیاده‌سازی شده و PostgreSQL به‌عنوان پایگاه داده اصلی و منبع مرجع اطلاعات در نظر گرفته شده است. مطابق الزامات پروژه، منطق دسترسی به داده‌های اصلی با کوئری‌های SQL مستقیم انجام می‌شود و در اغلب App‌ها ساختار لایه‌ای شامل Repository، Service، Serializer، View و فایل SQL به کار رفته است. این جداسازی باعث شده مسئولیت دریافت درخواست، اعتبارسنجی، منطق کسب‌وکار و دسترسی به پایگاه داده از یکدیگر مستقل باشند.

سامانه شامل App‌های accounts، catalog، tickets، reservations، payments، reports، support است. جریان اصلی سیستم از احراز هویت کاربر آغاز شده، با جستجوی رویداد و بلیت ادامه می‌یابد، سپس صندلی برای مدت محدود رزرو می‌شود و در صورت پرداخت موفق، موجودی صندلی به حالت فروخته‌شده تغییر کرده و بلیت نهایی صادر می‌شود. عملیات حساس مانند رزرو، پرداخت و لغو با Transaction و قفل سطری کنترل می‌شوند تا ناسازگاری داده و خرید همزمان یک صندلی رخ ندهد.

برای داده‌های موقت و کاهش بار PostgreSQL از Redis استفاده شده است. OTP، محدودیت ارسال مجدد و تعداد تلاش‌ها در Redis نگهداری می‌شوند و پروفایل کاربر و نتایج جستجوی معمولی بلیت نیز با TTL مشخص Cache می‌شوند. برای جستجوی پیشرفته، Elasticsearch به‌صورت یک سرویس مستقل استفاده شده و Index با نام tickets_search بر اساس ticket_category_id ساخته می‌شود. داده‌های Elasticsearch ابتدا با یک Management Command از PostgreSQL همگام می‌شوند و سپس تغییرات رزرو، پرداخت، لغو و آزادسازی موجودی بعد از Commit موفق تراکنش به‌صورت افزایشی Sync می‌شوند.

کلیدواژه‌ها: REST API، Elasticsearch، Redis، PostgreSQL، Django، رزرو بلیت، JWT، Cache، Transaction

فهرست مطالب

عنوان	بخش
مقدمه	فصل ۱
طراحی پایگاه داده و لایه دسترسی به داده	فصل ۲
طراحی و پیاده‌سازی بک‌اند	فصل ۳
پیاده‌سازی Redis و مکانیزم Cache	فصل ۴
پیاده‌سازی Elasticsearch و جستجوی پیشرفته	فصل ۵
سناریوهای عملکرد و آزمون	فصل ۶
نتیجه‌گیری	فصل ۷
راهنمای اجرا و سرویس‌ها	پیوست الف
خلاصه API‌های اصلی	پیوست ب

فهرست شکل‌ها

عنوان	شماره شکل
معماری کلی سامانه و ارتباط سرویس‌های داده	شکل ۱-۲
روابط سطح بالای موجودیت‌های پایگاه داده	شکل ۲-۲

عنوان	شماره شکل
معماری لایه‌ای داخل App های بک‌اند	شکل ۳-۱
چرخه رزرو، پرداخت، صدور و لغو بلیت	شکل ۳-۲
نقش Redis در OTP و Cache	شکل ۴-۱
جریان همگام‌سازی PostgreSQL و Elasticsearch	شکل ۵-۱

فهرست جدول‌ها

عنوان	شماره جدول
فناوری‌های اصلی پروژه	جدول ۲-۱
گروه‌های اصلی جداول پایگاه داده	جدول ۲-۲
ساختار App های بک‌اند	جدول ۳-۱
وظیفه App های اصلی	جدول ۳-۲
TTL ها و کاربردهای Redis	جدول ۴-۱
فیلدهای مهم Index جستجو	جدول ۵-۱
سناریوهای اصلی آزمون	جدول ۶-۱
خلاصه مسیرهای API	جدول ب-۱

۱-۱ بیان مسئله

سامانه رزرو بلیت مسابقات ورزشی تنها یک مجموعه فرم و جدول نیست؛ بخش‌های مختلف آن باید به‌صورت هماهنگ وضعیت رویداد، صندلی، رزرو، پرداخت و بلیت را مدیریت کنند. مهمترین چالش این است که چند کاربر ممکن است همزمان برای یک رویداد جستجو انجام دهند و حتی یک صندلی را انتخاب کنند. در نتیجه بک‌اند باید علاوه بر ارائه API‌های مناسب، سازگاری داده را در سطح پایگاه داده تضمین کند و اجازه ندهد یک موجودی به شکل همزمان به چند رزرو یا بلیت اختصاص پیدا کند.

از طرف دیگر، همه داده‌ها ماهیت یکسانی ندارند. اطلاعات کاربر، رویداد، رزرو و پرداخت باید به‌صورت پایدار در پایگاه داده رابطه‌ای ثبت شوند؛ اما داده‌هایی مانند OTP یا Cache جستجو کوتاه‌عمر هستند. همچنین جستجوی متنی و ترکیبی روی نام تیم‌ها، ورزشگاه، شهر و سایر مشخصات در یک موتور جستجو مانند Elasticsearch انعطاف بیشتری نسبت به Query‌های معمولی SQL دارد. بنابراین معماری پروژه بر استفاده همزمان از PostgreSQL، Redis، Elasticsearch بنا شده است.

۲-۱ اهداف و دستاوردهای پروژه

- طراحی مدل رابطه‌ای برای کاربران، رویدادها، سالن‌ها، صندلی‌ها، دسته‌های بلیت، رزرو، پرداخت، کیف پول، گزارش و پشتیبانی.
- پیاده‌سازی REST API با Django و Django REST Framework و تفکیک مسئولیت‌ها در لایه‌های مختلف.
- اجرای کوئری‌های اصلی دیتابیس به‌صورت Raw SQL مطابق الزام پروژه و عدم وابستگی منطق دامنه به ORM.
- کنترل همزمانی در رزرو و پرداخت با transaction.atomic و قفل‌های FOR UPDATE.
- پیاده‌سازی احراز هویت مبتنی بر JWT و OTP و کنترل دسترسی برای کاربران پشتیبانی.
- استفاده از Redis برای OTP، Profile Cache و Ticket Search Cache.
- پیاده‌سازی Advanced Search با Elasticsearch و همگام‌سازی داده جستجو با PostgreSQL.
- پوشش جریان کامل جستجو تا رزرو، پرداخت، صدور بلیت، لغو، بازپرداخت و گزارش مشکل.

۳-۱ دامنه و الزامات

صورت پروژه بر طراحی پایگاه داده تا سطح نرمال سوم، تعریف کلیدها و محدودیت‌های مناسب، استفاده از SQL مستقیم در بک‌اند، ذخیره OTP در Redis و ارائه API‌های تک‌مسئولیتی تأکید دارد. در پیاده‌سازی حاضر PostgreSQL دیتابیس اصلی است، Django سرویس HTTP و منطق کسب‌وکار را اجرا می‌کند، Redis برای داده موقت و Cache استفاده می‌شود و Elasticsearch مسئول جستجوی پیشرفته است. PostgreSQL در محیط توسعه روی پورت 5432 قرار دارد؛ Redis به‌صورت سرویس Docker روی پورت 6379 اجرا می‌شود و Elasticsearch نیز در محیط فعلی روی پورت 9270 در دسترس است.

۴-۱ ساختار گزارش

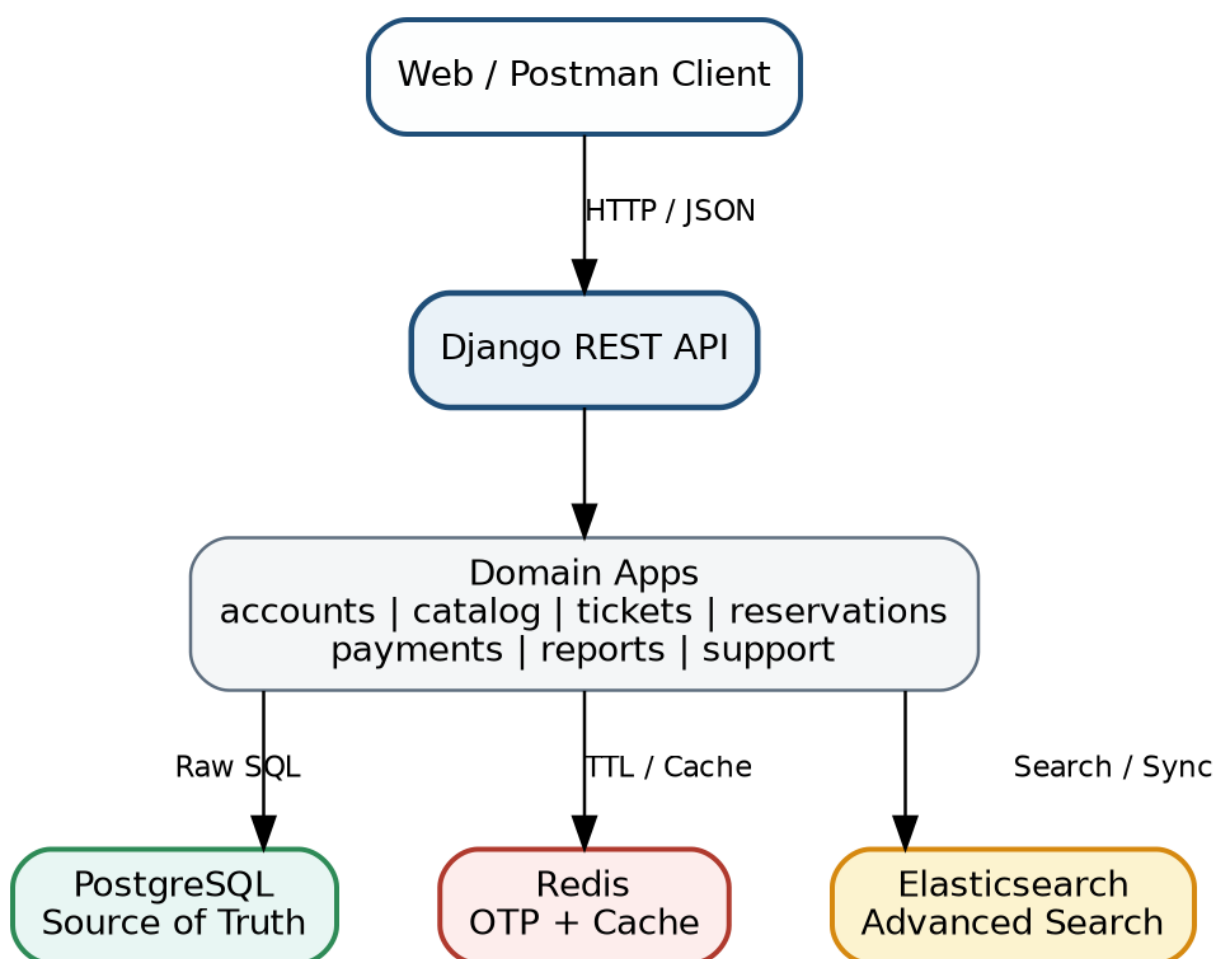
فصل دوم ساختار داده و نحوه دسترسی مستقیم به PostgreSQL را توضیح می‌دهد. فصل سوم معماری بک‌اند و نقش هر App را تشریح می‌کند. فصل چهارم به Redis و سیاست‌های TTL و Cache اختصاص دارد. فصل پنجم نحوه ساخت Index، Advanced Search و همگام‌سازی Elasticsearch را بیان می‌کند. فصل ششم سناریوهای اجرایی و آزمون‌های یکپارچگی را مرور می‌کند و فصل هفتم جمع‌بندی و مسیرهای توسعه آینده را ارائه می‌دهد.

۱-۲ معماری داده و فناوری‌های اصلی

هسته سامانه بر یک معماری چندمخزنی بنا شده است؛ با این حال فقط PostgreSQL منبع مرجع یا Source of Truth محسوب می‌شود. Redis و Elasticsearch داده‌های مشتق‌شده یا کوتاه‌عمر را نگهداری می‌کنند و در صورت از دست رفتن محتوای آنها، داده اصلی از PostgreSQL قابل بازسازی است. این تصمیم باعث می‌شود سازگاری تراکنشی در یک نقطه حفظ شود و سرویس‌های جانبی صرفاً برای کارایی و جستجو به کار روند.

فناوری	نقش در پروژه	پیکربندی محیط توسعه
PostgreSQL	نخیره دائمی کاربران، رویداد، موجودی، رزرو، پرداخت، بلیت، کیف پول و گزارش	localhost:5432 / sport_ticketing_dev
Redis	OTP، محدودیت‌های زمانی و Cache اطلاعات پرتکرار	localhost:6379 / DB 0
Elasticsearch	جستجوی پیشرفته و متنی روی اطلاعات قابل فروش	localhost:9270 / tickets_search
Django + DRF	API، اعتبارسنجی، منطق کسب‌وکار و هماهنگ‌سازی سرویس‌ها	Python 3.12 / Django 6.0.x

جدول ۲-۱ فناوری‌های اصلی پروژه



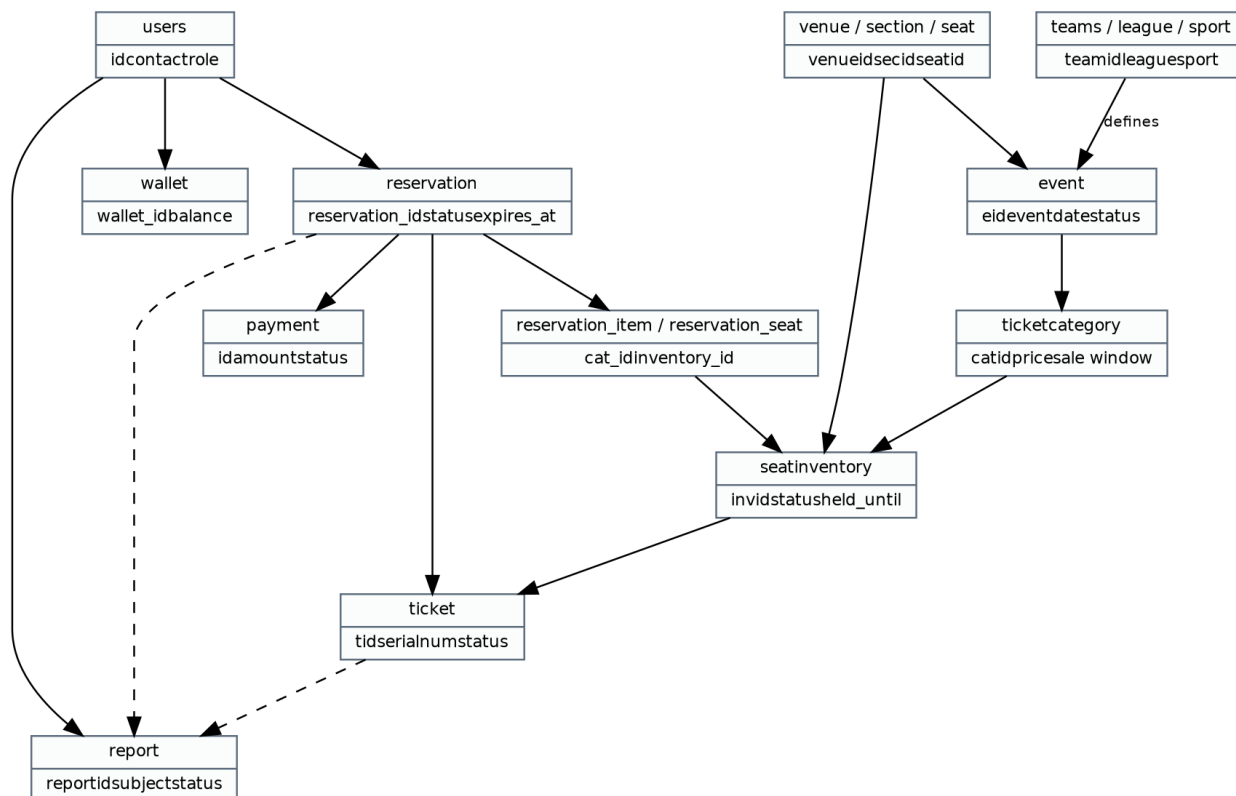
شکل ۲-۱ معماری کلی سامانه و ارتباط سرویس‌های داده

۲-۲ گروه‌های اصلی جداول

ساختار پایگاه داده از چند گروه منطقی تشکیل شده است. جداول پایه جغرافیا و کاتالوگ اطلاعاتی را نگهداری می‌کنند؛ جداول رویداد و ورزش مشخصات مسابقه را می‌سازند؛ جداول صندلی و موجودی وضعیت فروش را مدیریت می‌کنند و جداول تراکنشی فرآیند رزرو تا پرداخت و صدور بلیت را ثبت می‌کنند.

گروه	جداول یا موجودیت‌های شاخص	کاربرد
کاربر و دسترسی	users, wallet	مشخصات حساب، نقش کاربر و موجودی کیف پول
جغرافیا و محل برگزاری	country, province, cities, venue, venuesection, seat	مدل‌سازی موقعیت، ورزشگاه، بخش و صندلی
ورزش و مسابقه	sport, league, teams, league_team, organizer, event	نوع ورزش، لیگ، تیم‌ها، برگزارکننده و رویداد
فروش بلیت	ticketcategory, seatinventory	دسته قیمت، بازه فروش و وضعیت available/held/sold
رزرو و پرداخت	reservation, reservation_item, reservation_seat, payment	رزرو موقت، اقلام رزرو، اتصال صندلی و پرداخت
بلیت و کنترل	ticket, checkinlog, cancellation_rule	صدور بلیت، استفاده از بلیت و قوانین لغو
گزارش و پشتیبانی	report	ثبت مشکل کاربر و پیگیری توسط نقش support

جدول ۲-۲ گروه‌های اصلی جداول پایگاه داده



شکل ۲-۲ روابط سطح بالای موجودیت‌های پایگاه داده

۳-۲ وضعیت صندلی و رزرو

جدول `seatinventory` نقش محوری در جلوگیری از فروش تکراری دارد. هر موجودی در حالت عادی `available` است. پس از ایجاد رزرو، وضعیت آن به `held` تغییر کرده و `reservation_id`، `reservekey` و `held_until` برای آن ثبت می‌شود. اگر رزرو در بازه مجاز پرداخت شود، موجودی `sold` می‌شود. در صورت انقضای رزرو یا لغو مجاز بلیت، موجودی دوباره `available` می‌شود. به این ترتیب موجودی فیزیکی صندلی از رکورد رزرو و بلیت جدا نگه داشته شده و چرخه وضعیت آن قابل کنترل است.

۴-۲ نرمال‌سازی و یکپارچگی

اطلاعات تکرارشونده مانند نام شهر، تیم یا ورزشگاه داخل رکوردهای رزرو و بلیت تکرار نشده و از طریق کلیدهای خارجی به موجودیت اصلی متصل می‌شوند. رابطه چندبه‌چند لیگ و تیم در جدول واسط `league_team` و رابطه رزرو با صندلی‌های انتخاب‌شده در `reservation_seat` مدیریت می‌شود. همچنین قیمت خرید در `reservation_item` ذخیره می‌شود تا حتی اگر قیمت دسته بلیت در آینده تغییر کند، مبلغ تاریخی رزرو قابل بازیابی باشد. این تفکیک با اصول نرمال‌سازی سازگار بوده و احتمال ناسازگاری ناشی از تکرار داده را کاهش می‌دهد.

۵-۲ دسترسی مستقیم به SQL

مطابق الزام درس، Query های اصلی به صورت مستقیم نوشته شده‌اند. در هر App، فایل `sql.py` شامل متن Query است و `repositories.py` با استفاده از `django.db.connection` و `cursor` آن را اجرا می‌کند. خروجی `cursor` به Dictionary تبدیل شده و به Service بازگردانده می‌شود. در نتیجه لایه Service از جزئیات اتصال به دیتابیس مستقل باقی می‌ماند و امکان بررسی مستقل Query ها نیز وجود دارد.

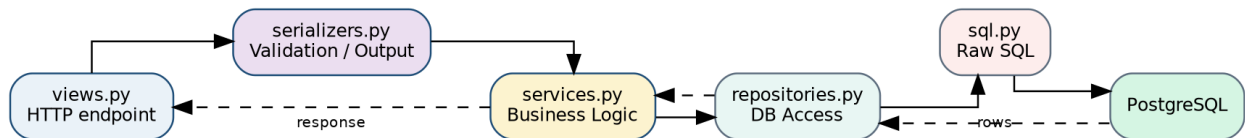
View -> Serializer -> Service -> Repository -> sql.py -> PostgreSQL
 <- result rows <-

۲-۶ تراکنش و کنترل همزمانی

عملیات رزرو، پرداخت و لغو بلیت چند تغییر وابسته را به صورت همزمان انجام می‌دهند؛ بنابراین با `transaction.atomic` اجرا می‌شوند. Query هایی که وضعیت حساس رزرو، صندلی، پرداخت یا کیف پول را می‌خوانند از `FOR UPDATE` استفاده می‌کنند تا رکورد تا پایان تراکنش قفل شود. برای نمونه در پرداخت، ابتدا رزرو و `inventory` های آن قفل و اعتبار `hold` بررسی می‌شود، سپس `payment` ثبت، `reservation` به `paid` و `inventory` به `sold` تغییر می‌کند و در نهایت `ticket` صادر می‌شود. اگر هر مرحله خطا دهد، همه تغییرات تراکنش `Rollback` می‌شوند.

۱-۳ معماری نرم‌افزار

بک‌اند به‌صورت ماژولار و بر اساس Domain App طراحی شده است. مسیر اصلی URL هر حوزه در فایل `urls.py` پروژه به App مربوطه متصل می‌شود: `accounts`، `catalog`، `tickets`، `reservations`، `payments`، `reports`، `support`. در داخل App ها مسئولیت‌ها تا حد امکان بین `View`، `Serializer`، `Service`، `Repository` و `SQL` جدا شده‌اند. `View` فقط ورودی HTTP را دریافت و خروجی JSON تولید می‌کند؛ `Serializer` مسئول اعتبارسنجی است؛ `Service` قوانین کسب‌وکار را اعمال می‌کند و `Repository` تنها نقطه اجرای `SQL` است.



شکل ۳-۱ معماری لایه‌ای داخل App های بک‌اند

۲-۳ ساختار ماژول‌ها

```

sport_ticketing/
manage.py
sport_ticketing_platform/
settings.py
urls.py
apps/
accounts/
catalog/
tickets/
reservations/
payments/
reports/
support/
    
```

App	مسئولیت اصلی	نمونه قابلیت‌ها
accounts	مدیریت هویت و حساب کاربر	Signup، Login، OTP، JWT، Profile تغییر، تماس و رمز
catalog	ارائه داده‌های مرجع	شهرها، ورزشگاه‌ها، ورزش‌ها، لیگ‌ها و تیم‌ها
tickets	جستجو و چرخه بلیت	Search، Details، My Bookings، Cancellation، Refund، Advanced Search
reservations	مدیریت رزرو موقت صندلی	Available Seats، Create Reservation، My Reservations، Expiration
payments	پرداخت و صدور بلیت	اعتبارسنجی رزرو، Wallet/روش پرداخت، Sold، Issue Ticket، کردن موجودی
reports	ثبت گزارش مشکل توسط کاربر	گزارش مرتبط با ticket یا reservation و issue_type
support	عملیات نقش پشتیبانی	گزارش‌ها، رزروها، بلیت‌های لغوشده و پرداخت‌های

App	مسئولیت اصلی	نمونه قابلیت‌ها
		مشکوک

جدول ۳-۱ ساختار و وظیفه App های اصلی

۳-۳ App حساب کاربری (accounts)

accounts نقطه ورود کاربر به سامانه است. API های جداگانه برای درخواست OTP ثبت نام، ثبت نام، درخواست OTP ورود، ورود، دریافت/ویرایش پروفایل، درخواست و تأیید تغییر اطلاعات تماس و تغییر رمز عبور در نظر گرفته شده اند. احراز هویت API های محافظت شده با JWT Authentication انجام می شود. داده OTP در PostgreSQL ذخیره نمی شود و Redis مسئول عمر کوتاه آن است که در فصل چهارم توضیح داده می شود.

۴-۳ App کاتالوگ (catalog)

catalog داده های مرجعی را که رابط کاربری برای ساخت فیلترها و فرم ها نیاز دارد ارائه می کند. مسیرهای cities، venues، sports، leagues و teams در این App قرار دارند. جدا کردن این حوزه از Tickets باعث می شود داده مرجع مستقل از عملیات فروش و رزرو قابل استفاده باشد.

۵-۳ App بلیت (tickets)

tickets دو نوع جستجو دارد. Search معمولی بر PostgreSQL اجرا شده و نتیجه آن در Redis Cache می شود. در کنار آن Advanced Search به Elasticsearch متصل است. جزئیات هر ticket category، فهرست بلیت های کاربر، اطلاعات مجاز بودن لغو و عملیات لغو نیز در همین App قرار دارند. در Cancellation ابتدا وضعیت بلیت، استفاده شدن یا نشدن آن، وضعیت reservation، شروع رویداد و قانون cancellation بررسی می شود؛ سپس مبلغ Refund محاسبه شده، کیف پول کاربر شارژ، ticket و reservation لغو، payment به refunded و inventory دوباره available می شود.

۶-۳ App رزرو (reservations)

reservations صندلی های قابل رزرو را واکنشی و عملیات Hold را انجام می دهد. مدت اعتبار رزرو در تنظیمات پروژه ۱۰ دقیقه است. هنگام ایجاد رزرو، inventory مورد نظر برای Update قفل می شود، فعال بودن صندلی، باز بودن بازه فروش و available بودن موجودی کنترل می شود و سپس reservation_item، reservation، reservation_seat و inventory ایجاد می گردند. در پایان inventory به held تغییر می کند. رزروهای منقضی شده توسط cleanup به expired تبدیل و inventory آنها آزاد می شود.

۷-۳ App پرداخت (payments)

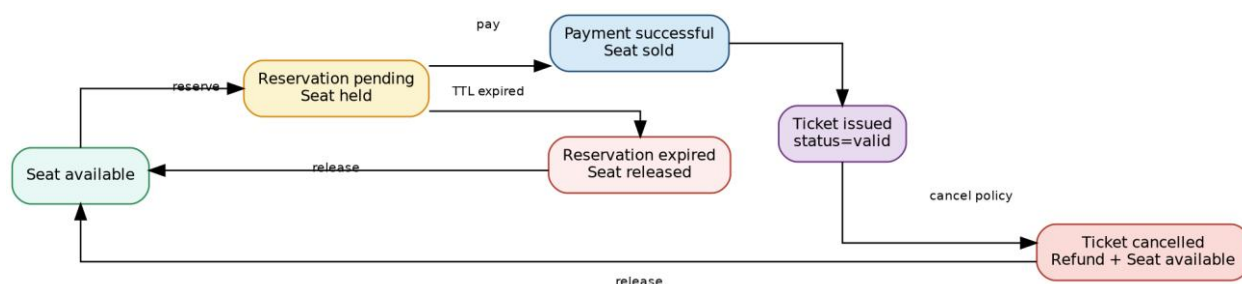
payments تنها رزروی را می پذیرد که متعلق به همان کاربر، در وضعیت pending و هنوز معتبر باشد. inventory های رزرو نیز باید همچنان held و متعلق به همان reservation باشند. در صورت انتخاب روش wallet، موجودی کیف پول بررسی و به صورت اتمیک کسر می شود. سپس payment موفق ثبت، reservation به inventory، paid به sold و برای صندلی ها ticket صادر می شود. قرار گرفتن همه این مراحل در یک تراکنش از ثبت پرداخت بدون صدور بلیت یا فروش صندلی بدون پرداخت جلوگیری می کند.

۸-۳ App های support و reports

کاربر احراز هویت شده می تواند برای بلیت یا رزرو خود یک report ثبت کند. issue_type از مجموعه ای کنترل شده مانند payment_problem، seat_problem، pricing_problem و سایر موارد انتخاب می شود. Service ابتدا مالکیت ticket/reservation را بررسی می کند و سپس report در وضعیت pending ایجاد می شود.

support با Permission اختصاصی IsSupport محافظت می شود و فقط کاربری با role برابر support اجازه دسترسی دارد. این App فهرست بلیت های لغو شده، پرداخت های مشکوک، گزارش ها و رزروها را ارائه می کند. پشتیبان می تواند گزارش را پاسخ دهد یا وضعیت رزرو pending را approve، modify یا cancel کند. Query پرداخت های مشکوک نیز ناسازگاری هایی مانند اختلاف مبلغ پرداخت و رزرو یا وضعیت های نامعتبر بین payment و reservation را شناسایی می کند.

۹-۳ مسیر انتها به انتهای خرید

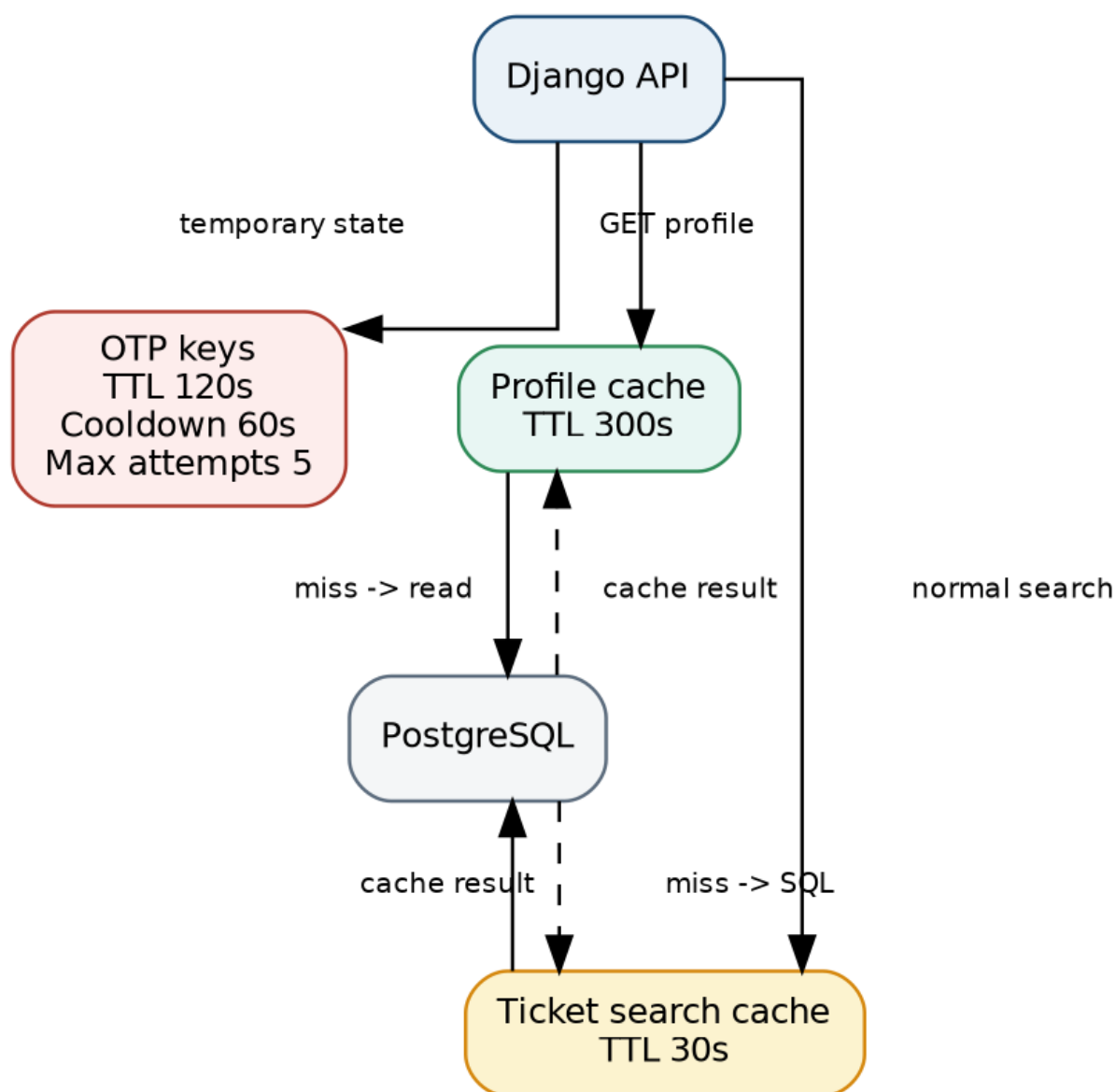


شکل ۳-۲ چرخه رزرو، پرداخت، صدور و لغو بلیت

در جریان اصلی، کاربر پس از ورود یک ticket category را جستجو و صندلی قابل دسترس را انتخاب می‌کند. رزرو موقت ایجاد و صندلی held می‌شود. پرداخت موفق وضعیت را به sold تبدیل کرده و ticket معتبر صادر می‌کند. اگر reservation قبل از پرداخت منقضی شود، صندلی آزاد می‌شود. اگر کاربر بعد از خرید و مطابق cancellation_rule بلیت را لغو کند، refund به کیف پول برگردانده و صندلی دوباره available می‌شود.

۴-۱ جایگاه Redis در معماری

Redis برای داده‌هایی انتخاب شده که یا عمر کوتاه دارند یا خواندن آنها پرتکرار است. سرویس Redis در محیط توسعه به‌صورت Docker اجرا شده و Django از طریق localhost:6379 و database شماره 0 به آن متصل می‌شود. PostgreSQL همچنان منبع اصلی است و حذف Redis نباید باعث از بین رفتن داده دائمی شود.



شکل ۴-۱ نقش Redis در OTP و Cache

کاربرد	TTL / محدودیت	رفتار
OTP	120 ثانیه	پس از پایان TTL کلید خودکار منقضی می‌شود
ارسال مجدد OTP	Cooldown برابر 60 ثانیه	از درخواست پشت سر هم کد جلوگیری می‌کند
تلاش OTP	حداکثر 5 تلاش	تعداد پاسخ‌های اشتباه در Redis کنترل می‌شود
Profile Cache	300 ثانیه	در Cache Hit پاسخ بدون Query مجدد PostgreSQL برمی‌گردد

رفتار	TTL / محدودیت	کاربرد
نتیجه جستجوی معمولی کوتاهمدت Cache می‌شود	30 ثانیه	Ticket Search Cache

جدول ۴-۱ TTL ها و کاربردهای Redis

۲-۴ OTP

برای Login، Signup و تغییر اطلاعات تماس، OTP با یک کلید شامل نوع عملیات و شناسه کاربر/ایمیل/شماره تماس در Redis ذخیره می‌شود. SETEX یا معادل آن TTL را همزمان با مقدار ثبت می‌کند. کلید Cooldown مانع ارسال فوری کد جدید می‌شود و یک کلید Attempts نیز تعداد ورود اشتباه کد را نگهداری می‌کند. پس از تأیید موفق یا عبور از حد مجاز تلاش، کلیدهای مربوط پاک می‌شوند. این طراحی مناسبتر از جدول دائمی OTP در PostgreSQL است زیرا داده پس از چند دقیقه ارزش نگهداری ندارد.

۳-۴ Cache پروفایل

در درخواست Profile ابتدا کلید <profile:user:<user_id در Redis بررسی می‌شود. در Cache Hit اطلاعات مستقیم برگردانده می‌شود. در Cache Miss داده از PostgreSQL خوانده و برای 300 ثانیه Cache می‌شود. هنگام ویرایش پروفایل، Cache قبلی invalidate یا با مقدار جدید جایگزین می‌شود تا کاربر داده کهنه دریافت نکند.

۴-۴ Cache جستجوی بلیت

Search معمولی بر اساس Filterهای ورودی یک Cache Key می‌سازد. اگر Query مشابه در 30 ثانیه اخیر اجرا شده باشد، نتیجه از Redis بازگردانده می‌شود؛ در غیر این صورت Query SQL TicketRepository را اجرا و نتیجه را Cache می‌کند. TTL کوتاه انتخاب شده چون available_count و وضعیت فروش می‌تواند سریع تغییر کند. علاوه بر TTL، بعد از عملیاتی‌هایی که ظرفیت را تغییر می‌دهند مانند Cancellation، Payment، Reservation و بعضی عملیات Support، Search Cache invalidate می‌شود.

۵-۴ سیاست سازگاری

اصل طراحی این است که Cache هیچگاه مرجع نهایی تصمیمگیری نباشد. حتی اگر نتیجه Search از Redis برگردد، هنگام رزرو واقعی دوباره وضعیت inventory در PostgreSQL و داخل Transaction بررسی می‌شود. بنابراین نمایش کوتاهمدت یک نتیجه Cache شده نمی‌تواند به فروش دوباره صندلی منجر شود؛ قفل و شرطهای SQL در مرحله رزرو آخرین تصمیم را می‌گیرند.

۵-۱ انگیزه استفاده از Elasticsearch

Search معمولی برای Filterهای مشخص مانند sport_id, team_id, city_id, venue_id با PostgreSQL مناسب است. اما در بخش Advanced Search نیاز به جستجوی متنی آزاد روی نام تیم‌ها، ورزشگاه، شهر، ورزش، لیگ و بخش وجود دارد. برای این هدف Elasticsearch اضافه شد تا multi_match و ترکیب Full-text Search با Filterهای عددی و زمانی را مستقل از Search معمولی فراهم کند.

۵-۲ ساختار Index

Index اصلی با نام tickets_search ساخته می‌شود. برخلاف بلیت نهایی که بعد از پرداخت صادر می‌شود، واحد مناسب برای Search قبل از خرید ticket category است؛ بنابراین id_ هر Document برابر ticket_category_id قرار داده شده است. یک Document علاوه بر شناسه دسته بلیت، اطلاعات مورد نیاز UI را نیز به صورت Denormalized نگهداری می‌کند تا Advanced Search بدون Joinهای پایگاه داده پاسخ دهد.

نوع کاربرد	نمونه فیلدها	گروه فیلد
فیلتر زمانی و وضعیت	event_id, event_datetime, event_status	رویداد
فیلتر و جستجوی متنی	sport_id/name, league_id/name/season	ورزش و لیگ
جستجوی مکان و Filter ID	venue, city, province, country	مکان
team filter و Keyword	home_team_id/name, away_team_id/name	تیم‌ها
قیمت و بازه فروش	ticket_category_id/name, current_price, sales_start/end	فروش
نمایش و حذف موارد بدون موجودی	section_id/name, section_type, available_count	بخش و ظرفیت

جدول ۵-۱ فیلدهای مهم Index جستجو

۵-۳ API جستجوی پیشرفته

مسیر Search معمولی بدون تغییر باقی مانده و Advanced Search در Endpoint جداگانه ارائه شده است. ورودی Advanced Search توسط Serializer مخصوص اعتبارسنجی می‌شود و می‌تواند شامل keyword, sport_id, team_id, city_id, venue_id, ticket_category_id، بازه تاریخ، بازه قیمت، limit و offset باشد. برای keyword از multi_match روی فیلدهای متنی و برای ID و قیمت از term/range filter استفاده می‌شود. خروجی با همان Serializer نتیجه Search معمولی هماهنگ شده تا Frontend یک ساختار پاسخ ثابت دریافت کند.

```
GET /api/tickets/search/ -> Redis -> PostgreSQL
GET /api/tickets/advanced-search/ -> Elasticsearch
```

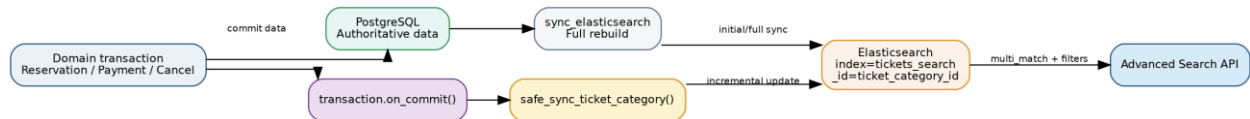
۵-۴ Initial Sync

Elasticsearch به صورت خودکار از جداول PostgreSQL آگاه نمی‌شود؛ بنابراین یک Django Management Command با نام sync_elasticsearch ایجاد شده است. این دستور Index را بازسازی می‌کند، داده‌های قابل جستجو را از PostgreSQL در چند

Batch می‌خواند و برای هر ticket_category یک Document ایجاد می‌کند. این فرآیند در راه‌اندازی اولیه، بعد از تغییر Mapping یا برای بازیابی Index استفاده می‌شود.

```
python manage.py sync_elasticsearch
```

۵-۵ همگامسازی افزایشی



شکل ۵-۱ جریان همگامسازی PostgreSQL و Elasticsearch

برای تغییرات روزمره، Full Sync مناسب نیست. بنابراین تابع sync_ticket_category فقط یک دسته بلیت را دوباره از PostgreSQL می‌خواند. اگر فروش باز باشد و available_count بیشتر از صفر باشد، Document با es.index ایجاد یا جایگزین می‌شود؛ در غیر این صورت Document از Index حذف می‌شود. به این ترتیب وقتی آخرین صندلی رزرو می‌شود، نتیجه از Search حذف و وقتی صندلی دوباره آزاد شود، Document قابل ایجاد مجدد است.

فراخوانی Sync بعد از transaction.on_commit ثبت می‌شود. در نتیجه Elasticsearch فقط بعد از Commit موفق PostgreSQL به‌روزرسانی می‌شود و اگر Transaction Rollback شود، موتور جستجو تغییر اشتباه دریافت نمی‌کند. wrapper امن safe_sync_ticket_category نیز خطای ارتباط Elasticsearch را Log می‌کند تا قطع موتور جستجو باعث شکست تراکنش اصلی رزرو یا پرداخت نشود.

۵-۶ نمونه داده Index

در تست اولیه، Index شامل دو Document برای دسته‌های Regular و VIP از یک رویداد بود. هر دو Document اطلاعات مسابقه Persepolis و Esteghlal در Azadi Stadium را نگهداری می‌کردند و available_count اولیه آنها 5 بود. این خروجی نشان داد Mapping، Full Sync و شناسه‌گذاری بر اساس ticket_category_id به‌درستی انجام شده است.

۶-۱ محیط آزمون

API ها با Postman و در محیط توسعه محلی تست شدند. Django روی یک پورت محلی، PostgreSQL روی 5432، Redis روی 6379 و Elasticsearch روی 9270 در دسترس بودند. برای تست جستجو یک Dataset شامل رویداد فوتبال، تیم‌های Persepolis و Esteghlal، ورزشگاه Azadi و دو دسته بلیت VIP و Regular استفاده شد. هدف اصلی آزمون‌ها فقط دریافت HTTP 200 نبود، بلکه بررسی تغییر صحیح Status ها و هماهنگی Redis، PostgreSQL و Elasticsearch بود.

انتظار	عملیات اصلی	سناریو
اعتبار TTL، محدودیت تلاش و دسترسی به API محافظت شده	Request OTP -> Signup/Login -> JWT	احراز هویت
بار دوم از Redis Cache پاسخ داده شود	اجرای فیلتر مشابه دو بار	Search معمولی
reservation=pending و inventory=held و available_count کاهش یابد	رزرو inventory available	Reservation
reservation=paid، ticket صادر و inventory=sold شود	پرداخت رزرو معتبر	Payment
reservation=expired و inventory=available شود	تمام شدن TTL رزرو	Expiration
wallet، Refund به payment=refunded، ticket=cancelled و inventory آزاد شود	لغو بلیت طبق rule	Cancellation
نتیجه از Elasticsearch با ساختار استاندارد API برگردد	keyword و Filter های ترکیبی	Advanced Search
کاربر عادی 403 و support مجاز به مشاهده/به‌روزرسانی باشد	گزارش/رزرو با role support	Support

جدول ۶-۱ سناریوهای اصلی آزمون

۶-۲ آزمون همزمانی و Transaction

در مسیر Reservation و Query، Payment های FOR UPDATE برای جلوگیری از Race Condition اهمیت دارند. درخواست همزمان ممکن است قبل از رزرو یک صندلی، آن را available مشاهده کنند؛ اما فقط تراکنشی که زودتر قفل row را به دست می‌آورد قادر به تغییر وضعیت است و درخواست دوم بعد از آزاد شدن قفل دوباره شرایط را بررسی خواهد کرد. در پرداخت نیز قفل reservation و inventories مانع از پرداخت تکراری یا تغییر همزمان وضعیت موجودی می‌شود.

۳-۶ آزمون Redis

در تست OTP با redis-cli می‌توان کلیدهای *:otp، مقدار کد، TTL، کلید cooldown و attempts را مشاهده کرد. در Profile Cache نیز بعد از اولین GET کلید <profile:user:<id ساخته می‌شود و درخواست بعدی تا پایان TTL از Redis پاسخ می‌گیرد. در Search Cache نیز با ایجاد رزرو و تغییر invalidate، available_count باعث حذف نتیجه قدیمی می‌شود تا Search بعدی مجدداً از PostgreSQL خوانده شود.

۴-۶ آزمون Elasticsearch

پس از اجرای sync_elasticsearch، درخواست cat/indices_ وجود tickets_search را نشان داد و match_all دو Document موجود را برگرداند. سپس می‌توان قبل و بعد از Reservation مقدار available_count یک Document را مقایسه کرد؛ مقدار 5 باید پس از Hold یک صندلی به 4 تبدیل شود. Payment مقدار را روی 4 نگه می‌دارد زیرا held و sold هر دو از available خارج‌اند، و Cancellation یا Expiration دوباره آن را افزایش می‌دهد. همچنین اگر available_count به صفر برسد Document از Index حذف می‌شود.

۵-۶ رفتار در خرابی سرویس جانبی

Redis و Elasticsearch سرویس‌های کمکی هستند و قطع آنها نباید داده مرجع PostgreSQL را خراب کند. برای Elasticsearch، Sync افزایشی از wrapper امن استفاده می‌کند؛ بنابراین اگر سرویس در دسترس نباشد، خطا Log شده ولی تراکنش اصلی که قبلاً Commit شده است باقی می‌ماند. پس از بازیابی سرویس می‌توان Management Command را اجرا کرد تا Index دوباره از Source of Truth ساخته شود. برای Redis نیز از دست رفتن Cache به معنی Cache Miss است و داده قابل بازخوانی از PostgreSQL خواهد بود؛ تنها قابلیت‌های کاملاً وابسته به داده موقت مانند OTP تا بازگشت Redis در دسترس نیستند.

۶-۶ محدودیت‌های نسخه فعلی

- فرآیند پرداخت پروژه آموزشی است و به درگاه بانکی واقعی متصل نشده است.
- پاکسازی Reservation‌های منقضی‌شده در نسخه فعلی هنگام اجرای بعضی API‌ها انجام می‌شود؛ برای اجرای دقیق در لحظه می‌توان Scheduler یا Celery Beat اضافه کرد.
- Elasticsearch برای Search به صورت Denormalized داده نگه می‌دارد و سازگاری آن از نوع Eventual Consistency پس از Commit است.
- در محیط توسعه اطلاعات سرویس‌ها روی localhost قرار دارند؛ برای Deployment واقعی باید مقادیر اتصال و Secret‌ها به Environment Variable منتقل شوند.
- تست‌های بار و Concurrent Request گسترده می‌تواند در مرحله بعد با ابزارهایی مانند Locust یا JMeter تکمیل شود.

۱-۷ جمع‌بندی

در این پروژه یک بک‌اند یکپارچه برای سامانه رزرو و خرید بلیت ورزشی طراحی شد که منطق کسب‌وکار را به‌طور مستقیم به یک مدل داده رابطه‌ای متصل می‌کند. ساختار App محور Django باعث شده حوزه‌های حساب کاربری، کاتالوگ، جستجو، رزرو، پرداخت، گزارش و پشتیبانی از هم جدا باشند و در داخل هر حوزه نیز Repository، Service، Serializer، View و SQL مسئولیت‌های مشخصی داشته باشند.

PostgreSQL مرجع نهایی اطلاعات است و تراکنش‌ها و قفل‌های سطری سازگاری عملیات حساس را تضمین می‌کنند. Redis با نگهداری OTP و Cache داده‌های پرتکرار، تعداد Queryهای غیرضروری را کاهش می‌دهد. Elasticsearch نیز Advanced Search را از Search SQL جدا کرده و با Full Sync و Incremental Sync پس از Commit، داده قابل جستجو را با PostgreSQL همگام نگه می‌دارد. در نتیجه سه فناوری داده‌ای پروژه نقش‌های مکمل دارند و هیچ‌کدام جای دیگری را به‌طور کامل نمی‌گیرد.

۲-۷ پیشنهاد‌های توسعه

- اجرای Cleanup رزروهای منقضی‌شده به‌صورت دوره‌ای با Celery Beat یا یک Scheduler مستقل.
- انتقال تمام Connection String ها، کلیدهای JWT و تنظیمات حساس به Environment Variable و استفاده از تنظیمات جداگانه Development/Production.
- استفاده از Docker Compose برای اجرای یکپارچه Redis، PostgreSQL، Backend و Elasticsearch.
- افزودن Outbox Pattern یا Queue برای Sync قابل بازیابی‌تر بین PostgreSQL و Elasticsearch در مقیاس بزرگ.
- افزودن تست‌های خودکار Unit/Integration و تست بار برای بررسی Race Condition و رفتار سیستم تحت کاربران همزمان.
- افزودن Monitoring و Structured Logging برای مشاهده خطاهای Search، Cache و Transaction در محیط عملیاتی.

الف-۱ پیش‌نیازها

Python و Virtual Environment پروژه باید فعال باشند و Redis، PostgreSQL و Elasticsearch در دسترس باشند. دیتابیس توسعه با نام sport_ticketing_dev روی PostgreSQL پی‌کربندی شده است. Redis و Elasticsearch می‌توانند با Docker اجرا شوند. در گزارش هیچ رمز عبور، Secret Key یا Credential حساس درج نشده است.

الف-۲ اجرای سرویس‌ها

```
# Redis
docker start redis

# Elasticsearch (container name may differ)
docker start elasticsearch

# Test Elasticsearch
GET http://localhost:9270/

# Full Elasticsearch sync
python manage.py sync_elasticsearch

# Django development server
python manage.py runserver 127.0.0.1:8001
```

الف-۳ بررسی سرویس‌ها

```
# Redis CLI
docker exec -it redis redis-cli
SCAN 0 MATCH otp:* COUNT 100

# Elasticsearch indices
GET http://localhost:9270/_cat/indices?v

# Search all indexed documents
GET http://localhost:9270/tickets_search/_search
```

App	Method	Path	کاربرد
Accounts	POST	/api/accounts/signup/otp/request/	درخواست OTP ثبت نام
Accounts	POST	/api/accounts/signup/	ثبت نام
Accounts	POST	/api/accounts/login/otp/request/	درخواست OTP ورود
Accounts	POST	/api/accounts/login/	ورود و دریافت JWT
Accounts	GET/PATCH	/api/accounts/profile/	مشاهده/ویرایش پروفایل
Catalog	GET	/api/catalog/cities/	شهرها
Catalog	GET	/api/catalog/venues/	ورزشگاهها
Catalog	GET	/api/catalog/sports/	ورزشها
Catalog	GET	/api/catalog/leagues/	لیگها
Catalog	GET	/api/catalog/teams/	تیمها
Tickets	GET	/api/tickets/search/	جستجوی معمولی + SQL Redis
Tickets	GET	/api/tickets/advanced-search/	جستجوی پیشرفته Elasticsearch
Tickets	GET	/api/tickets/details/<ticket_category_id>/	جزئیات دسته بلیت
Tickets	GET	/api/tickets/my-bookings/	بلیت های کاربر
Tickets	GET	api/tickets/<ticket_id>/cancellation-/ /info	محاسبه شرایط لغو
Tickets	POST	/api/tickets/<ticket_id>/cancel/	لغو و Refund
Reservations	GET	/api/reservations/seats/	صندلی های قابل رزرو
Reservations	POST	/api/reservations/	ایجاد رزرو موقت
Reservations	GET	/api/reservations/mine/	رزرو های کاربر
Payments	POST	/api/payments/	پرداخت رزرو و صدور بلیت
Reports	POST	/api/reports/	ثبت گزارش مشکل
Support	GET	/api/support/cancelled-tickets/	بلیت های لغوشده
Support	GET	/api/support/suspicious-payments/	پرداخت های مشکوک
Support	GET/PATCH	.../api/support/reports/	بررسی و پاسخ گزارشها

App	Method	Path	کاربرد
Support	GET/PATCH	.../api/support/reservations/	مدیریت رزروهای نیازمند بررسی

جدول ب-۱ خلاصه مسیرهای API

منابع و مستندات

- صورت پروژه نهایی درس پایگاه داده‌ها، سامانه رزرو و خرید بلیت مسابقات ورزشی.
- Django و Django REST Framework - مستندات رسمی مربوط به Transaction، URL Routing و API.
- PostgreSQL - مستندات Transaction، SQL و Row-level Locking.
- Redis - مستندات SETEX، TTL و الگوهای Cache.
- Elasticsearch Python Client - مستندات Index، Mapping و Query DSL.